Dynamic Portfolio Allocation

An Iterative Analysis of Model Memory (V1-V3) Against Real-World Volatility

The Challenge: Static vs. Dynamic

The Problem: Traditional "buy & hold" strategies are passive. They ride markets up but are fully exposed to crashes.

The Goal: To build an intelligent PPO agent that *learns* a dynamic policy:

1. **Maximize** risk-adjusted returns (Sharpe Ratio).
2. **Learn** when to seek profit and when to flee to cash.

The Test: Can the agent beat static benchmarks (NIFTY50, Equal-Weight) in radically different market regimes?

Technical Stack & Libraries

Core ML & Reinforcement Learning:

- **PyTorch:** The backend tensor library for all neural network operations.
- **Stable-Baselines3:** The primary framework used to implement the **PPO** algorithm and manage training.
- **Gymnasium:** (Formerly OpenAI Gym) The toolkit used to define and run the custom StockPortfolioEnv environment.

Data Processing & Feature Engineering:

- **yfinance:** Used to download all historical stock data for the NIFTY50.
- **Pandas:** The core library for all data manipulation, cleaning, time-series alignment, and CSV I/O.
- **NumPy:** For high-performance numerical operations and array manipulation.
- **stockstats & ta:** Libraries used for the full suite of technical indicator calculations (MACD, RSI, ATR, SMAs, etc.).

Visualization & Backtesting:

- **Matplotlib:** Used to generate the final comparison plots of portfolio value over time.
- **pyfolio:** (Imported in notebooks) A library for in-depth portfolio performance and risk analysis.

Environment & Utilities:

- **Google Colab:** The notebook environment used for development and training.
- **google.colab.drive:** Used to mount Google Drive for persistent storage of data, models, and logs.
- **os & sys:** For path management and system interactions.
- **datetime:** For handling date ranges for data extraction.

The Contenders: An Evolution of "Memory"

V1: The "Memoryless" Agent

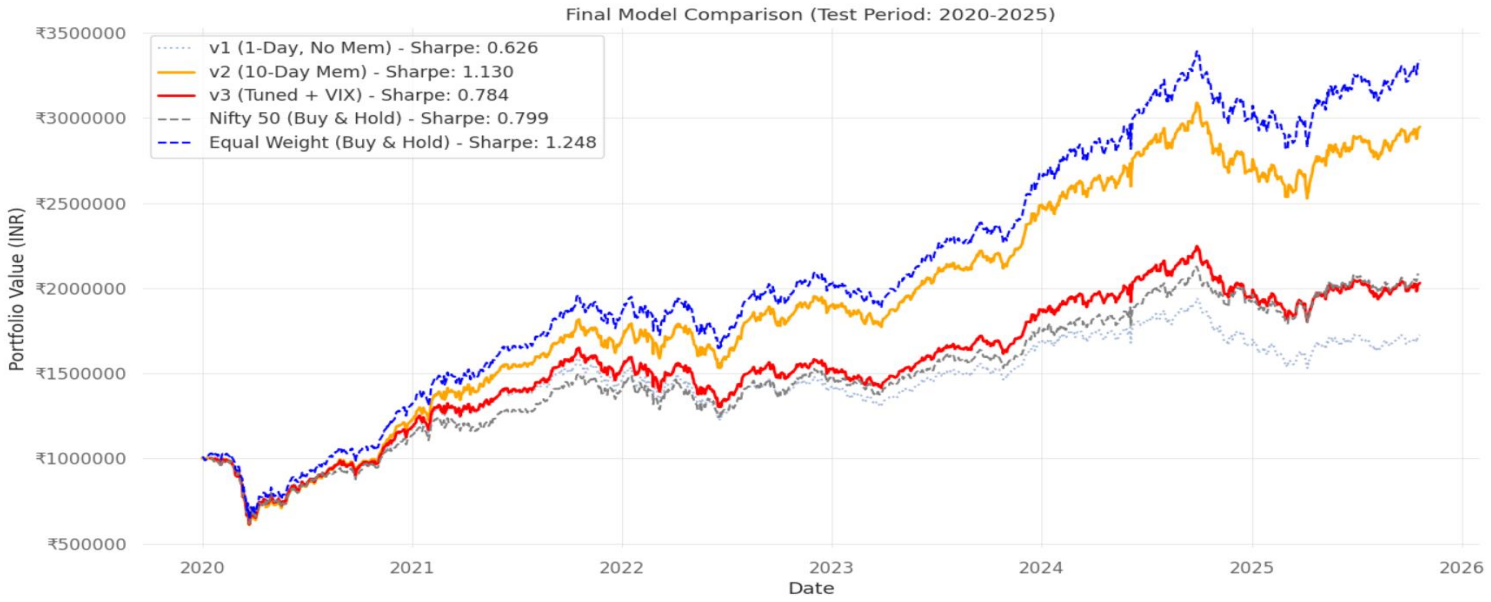
- time_window = 1
- Sees only today's data. It's purely reactive, trading on noise.

V2: The "Informed" Agent

- time_window = 10
- Sees a 10-day pattern. It can

V3: The "Over-Engineered" Agent

- time_window = 10 + *VIX/NIFTY features* + *Drawdown Penalty*
- An attempt to "force" risk management with more data and a penalty.



Trial by Fire: The 2020 "Black Swan"

- **Test:** The sudden, violent COVID-19 crash.
- **Verdict: Total Failure.** All models failed, but V2 was the worst.
- **Insight:** The V2 agent's 10-day memory was its "Achilles' heel." By the time its 10-day window confirmed the crash, the market had already bottomed out. **It was too slow.**

Model/Regieme (Sharpe Ratio)	**2020 Crash**
RL Model (V2)	**-1.470 (Worst)**
RL Model (V3)	-1.506
RL Model (V1)	-1.378
NIFTY50 (Benchmark)	-1.203
Equal-Weight (Benchmark)	**-1.114 (Best)**

Table depicting sharpe during COVID crash

Redemption: The 2022 "Slow Grind"

- **Test:** The year-long, protracted bear market.
- **Verdict: Decisive Victory.** The V2 agent's 10-day memory was its greatest strength.
- **Insight:** This slow-moving downturn was the *perfect* scenario for V2. It identified the bearish pattern, adapted its policy, and **defensively rotated to cash**, generating a positive return while all others struggled.

Model / Regime (Sharpe Ratio)	**2022 Bear Market**
🏆 **RL Model (V2)**	**+0.734 (Winner)**
Equal-Weight (Benchmark)	+0.592
NIFTY50 (Benchmark)	+0.335
RL Model (V3)	+0.049
RL Model (V1)	-0.123

Table depicting sharpe during 2022 Bear Regieme

The Verdict: A Specialist, Not a Silver Bullet

Conclusion:

1. **Memory is critical:** A 1-day window (V1) fails. A 10-day window (V2) learns.
2. **Simplicity is key:** The complex V3 model was "over-engineered" and failed. V2's simple architecture was superior.
3. **The V2 model is a *specialist*:** It learned a powerful defensive policy against **slow-moving bear markets**, but this same defense makes it **too slow for a "black swan" crash.**

Future Work:

1. **Tune the Specialist:** Hyper-tune the V2 model to optimize its winning strategy.
2. **Create a "Sprinter":** Explore a 3-day or 5-day time window to create an agent that can react to sudden crashes like 2020
3. **Deploy V2:** Paper-trade the V2 model to test its performance on live market data.